YEAR 12 · AI ENGINEERING · LECTURE 1

LangChain

Building blocks for apps powered by large language models

Calling an LLM once is easy. Building an app around it is the hard part.

TODAY

1. Why we need a framework
2. The building blocks
3. Prompts, models, parsers
4. Chains with the | pipe
5. Structured output
6. Memory
7. Tools and agents
8. Activity + quiz

Calling a model once is easy…

- One API call: send a question, get an answer back
- But a real app also needs:
  - the **same prompt** reused with different inputs
  - several steps in a row (summarise → translate → check)
  - answers as **data** a program can use, not just paragraphs
  - **memory** of the conversation
  - **tools**: search, calculators, databases, APIs
  - freedom to switch models (OpenAI, Claude, Gemini, local) without rewriting everything
- Writing all of this by hand for every app is slow and error-prone

TALKING POINTS

Start with a question to the class: "If you wanted to build a homework helper website on top of ChatGPT, what would you need besides the model?" Collect answers on the board — you'll usually get memory, a nice UI, safety, using the web. Then reveal the list. Key message: the model is only one part of an AI app; LangChain gives us standard parts for the rest.

What is LangChain?

- An **open-source framework** (Python and JavaScript), first released in **2022**
- Gives you **standard, swappable building blocks** for LLM applications
- Lets you **connect** models to prompts, data, memory and tools
- Part of a family:
  - **LangChain** — components and agents
  - **LangGraph** — workflows with branches and loops (Lecture 2)
  - **LangSmith** — tracing, testing and monitoring your app
- Hundreds of **integrations**: model providers, vector databases, document loaders, tools

TALKING POINTS

Analogy: LangChain is like Lego Technic for AI apps. Each piece has a standard connector, so a 'prompt' piece clicks onto a 'model' piece, which clicks onto an 'output parser' piece. If you swap the motor (the model) for a different brand, the rest of the model still works. Mention that it's free and open source — students can use it in their own projects.

The building blocks

Block	What it does	Everyday analogy
Chat model	Talks to any LLM with the same code (ChatOpenAI, ChatAnthropic, ChatOllama…)	A universal phone charger
Prompt template	A prompt with blanks: {subject}, {question}	A fill-in-the-blanks form
Output parser	Turns the model's text into a string, JSON or a Python object	A translator for your program
Chain (LCEL)	Joins blocks with the | pipe	A factory conveyor belt
Memory / history	Keeps the conversation so the model can refer back	A notebook of what was said
Retriever	Finds relevant documents for a question (Lecture 3)	A librarian
Tools + agents	Lets the model call functions and decide what to do next	An assistant with a toolbox

TALKING POINTS

Walk through the table row by row and keep the analogies — students remember them. Tell them we'll write code for the first five today; retrievers are the star of Lecture 3 (RAG), and agents get much more powerful with LangGraph in Lecture 2.

Block 1 — Chat models and messages

Every provider uses the **same interface**: `.invoke()`, `.stream()`, `.batch()`

```
from langchain_openai import ChatOpenAI          # or ChatAnthropic, ChatOllama ...
from langchain_core.messages import SystemMessage, HumanMessage

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.3)

reply = llm.invoke([
    SystemMessage("You only answer in rhyming couplets."),
    HumanMessage("What is a volcano?"),
])
print(reply.content)
```


TALKING POINTS

Point out the two message roles: the system message sets the rules, the human message is the question. Ask: "What would we change to use Claude instead?" Answer: only the import and the first line. That's the whole point of a standard interface. Model names change often — always check the provider's docs. A free local option is ChatOllama with an open model.

Block 2 — Prompt templates

Write the prompt **once**, fill in the blanks many times

```
from langchain_core.prompts import ChatPromptTemplate

explain = ChatPromptTemplate.from_messages([
    ("system", "You are a {subject} teacher. Explain for a {age}-year-old "
               "in at most {sentences} sentences."),
    ("user", "{question}"),
])

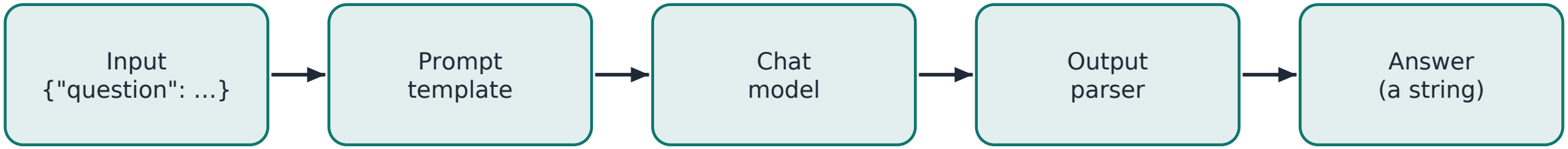
filled = explain.invoke({"subject": "physics", "age": 17,
                         "sentences": 2, "question": "Why do things fall?"})
print(filled.to_messages())
```


TALKING POINTS

Compare with a Google Docs template or a mail merge. Good prompts take effort to write, so we want to reuse them. Ask a student to suggest new values for subject/age/question and predict how the answer will change.

Block 3 — Chains with the | pipe (LCEL)

LCEL = LangChain Expression Language. a | b means "send the output of a into b".




```
chain = explain | llm | StrOutputParser()
chain.invoke({"subject": "chemistry", "age": 17, "sentences": 2, "question": "What is an
acid?"})
```

Every chain automatically gets `.invoke()` (one input), `.batch()` (many inputs at once) and `.stream()` (word by word).

TALKING POINTS

The pipe symbol comes from the Unix command line, where you chain small programs together. Draw the conveyor belt on the board: data goes in the left, each station does one job, the answer comes out the right. Emphasise that each block is small and testable on its own.

Chains can feed other chains

Sequential and parallel chains

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough, RunnableParallel

make_q = (ChatPromptTemplate.from_template("Write ONE quiz question about {topic}.")
          | llm | StrOutputParser())
answer_q = (ChatPromptTemplate.from_template("Answer in one sentence: {question}")
            | llm | StrOutputParser())

quiz = {"question": make_q} | RunnablePassthrough.assign(answer=answer_q)
print(quiz.invoke({"topic": "the water cycle"}))

both = RunnableParallel(simple=..., expert=...)   # run chains side by side
```


TALKING POINTS

Step through it: the first chain writes a question; RunnablePassthrough.assign keeps that question and adds an answer from the second chain. RunnableParallel runs several chains on the same input at the same time — e.g. an explanation for a 7-year-old and for a university student. Ask: what other two-step chains could a school use? (Summarise a notice → translate it; mark an answer → write feedback.)

Block 4 — Structured output: answers as data

Describe the **shape** you want with Pydantic; LangChain makes the model fill it in

```
from pydantic import BaseModel, Field

class Flashcard(BaseModel):
    term: str = Field(description="the key word")
    definition: str = Field(description="under 20 words")
    difficulty: int = Field(description="1 (easy) to 5 (hard)")

card = llm.with_structured_output(Flashcard).invoke(
    "Make a flashcard about the word 'evaporation'.")

print(card.term, "-", card.definition, "| level", card.difficulty)
```


TALKING POINTS

Programs can't easily use a paragraph of text, but they can use an object with fields. This is how you'd build a flashcard app, save results to a database, or show a table on a website. The Field descriptions are instructions to the model — they matter!

Block 5 — Memory: remembering the conversation

LLMs don't remember — the app stores the history and sends it every time

```
from langchain_core.prompts import MessagesPlaceholder
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a friendly maths tutor."),
    MessagesPlaceholder("history"),           # earlier messages go here
    ("user", "{input}")])

store = {}
def get_history(session_id):
    return store.setdefault(session_id, InMemoryChatMessageHistory())

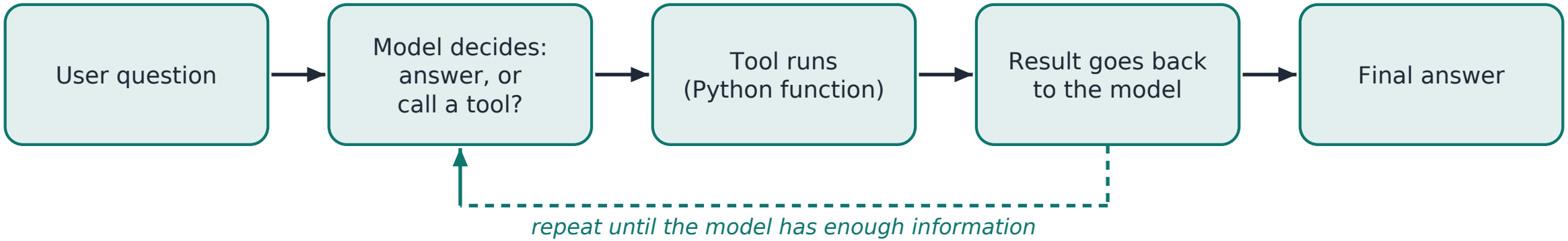
tutor = RunnableWithMessageHistory(prompt | llm | StrOutputParser(), get_history,
                                   input_messages_key="input", history_messages_key="history")
cfg = {"configurable": {"session_id": "student-1"}}
tutor.invoke({"input": "Hi, I'm Aroha."}, config=cfg)
tutor.invoke({"input": "What's my name?"}, config=cfg)     # -> "Aroha"
```


TALKING POINTS

Key misconception to fix: the model itself has no memory between calls. Memory is the app re-sending earlier messages. Consequence: long chats cost more tokens. Ask: what happens with session_id "student-2"? (A fresh conversation — that's how chat apps keep users separate.)

Block 6 — Tools and agents

A **tool** is a function the model is allowed to call. An **agent** is a model in a loop that decides which tools to use.



TALKING POINTS

Act it out: one student is the 'model', another holds a calculator (the 'tool'). The model can only ask for the calculator, not use it directly. That's exactly how tool calling works — the model writes a request, our code runs the function and returns the result.

Tools and agents in code

The **docstring** is the tool's description — the model reads it to decide when to use the tool

```
from langchain_core.tools import tool
from langchain.agents import create_agent

@tool
def term_dates(term: int) -> str:
    """Get the start and end dates of a school term (1 to 4)."""
    return {1: "3 Feb - 11 Apr", 2: "28 Apr - 4 Jul",
            3: "21 Jul - 26 Sep", 4: "13 Oct - 16 Dec"}.get(term, "No such term")

@tool
def multiply(a: float, b: float) -> float:
    """Multiply two numbers."""
    return a * b

agent = create_agent(llm, tools=[term_dates, multiply],
                     system_prompt="You are a school office assistant.")
agent.invoke({"messages": [{"role": "user",
              "content": "When does term 3 start? And what is 64 x 185?"}]})
```


TALKING POINTS

Highlight the docstrings: a vague description like 'does stuff' leads to the model picking the wrong tool. create_agent builds the loop from the previous slide for us. Next lecture we'll see that this agent is actually built with LangGraph underneath.

When should you use LangChain?

Good fit ✓	Maybe not ✗
Apps with several steps, templates and parsing	One single API call — just use the provider's SDK
You want to swap models or providers easily	You need every detail hand-tuned for speed
You need ready-made integrations (loaders, vector DBs, tools)	A tiny script where an extra library adds complexity
Prototyping quickly, then adding tracing with LangSmith	Complex branching workflows → use **LangGraph** (next lecture)

TALKING POINTS

Be balanced: frameworks save time but add a layer to learn and debug. Professional developers choose based on the job. Ask the class to place two example projects in the table: a flashcard generator (good fit) and a single 'translate this sentence' script (overkill).

Activity — design a chain (10 minutes, pairs)

- Pick one: **Revision-card maker** · **Newsletter translator** · **Essay feedback helper**
- On paper, draw your chain as boxes and arrows:
  - What is the **input**? Which **prompt template**(s) with which blanks?
  - Which **output** shape? (string, JSON, a Pydantic class — list its fields)
  - Does it need **memory**? Any **tools**?
- Write the one-line LCEL version, e.g. `prompt | llm | parser`
- Swap with another pair: can they spot a missing step or a risk?

TALKING POINTS

Circulate and push pairs to be specific about the output shape — that's where most designs are vague. Pick one or two designs to show on the board and turn into real code if you have time at the end.

Recap + exit quiz

- LangChain = **standard building blocks** for LLM apps: models, prompts, parsers, memory, retrievers, tools
- `prompt | llm | parser` is a **chain**; chains can run in sequence or in parallel
- `with_structured_output` turns answers into Python objects
- Memory = the app **re-sending history**; agents = models calling **tools** in a loop
- **Quiz:** 1) What does the | symbol do? 2) Where does a chat model's memory really live? 3) Why do tool docstrings matter? 4) Name one case where LangChain is overkill.

TALKING POINTS

Answers: 1) passes the output of the left block into the right block. 2) In the application, which re-sends the message history. 3) The model reads them to decide when and how to call the tool. 4) A single simple API call / tiny script. Preview: next lecture — LangGraph, for workflows that need branches, loops and pauses.